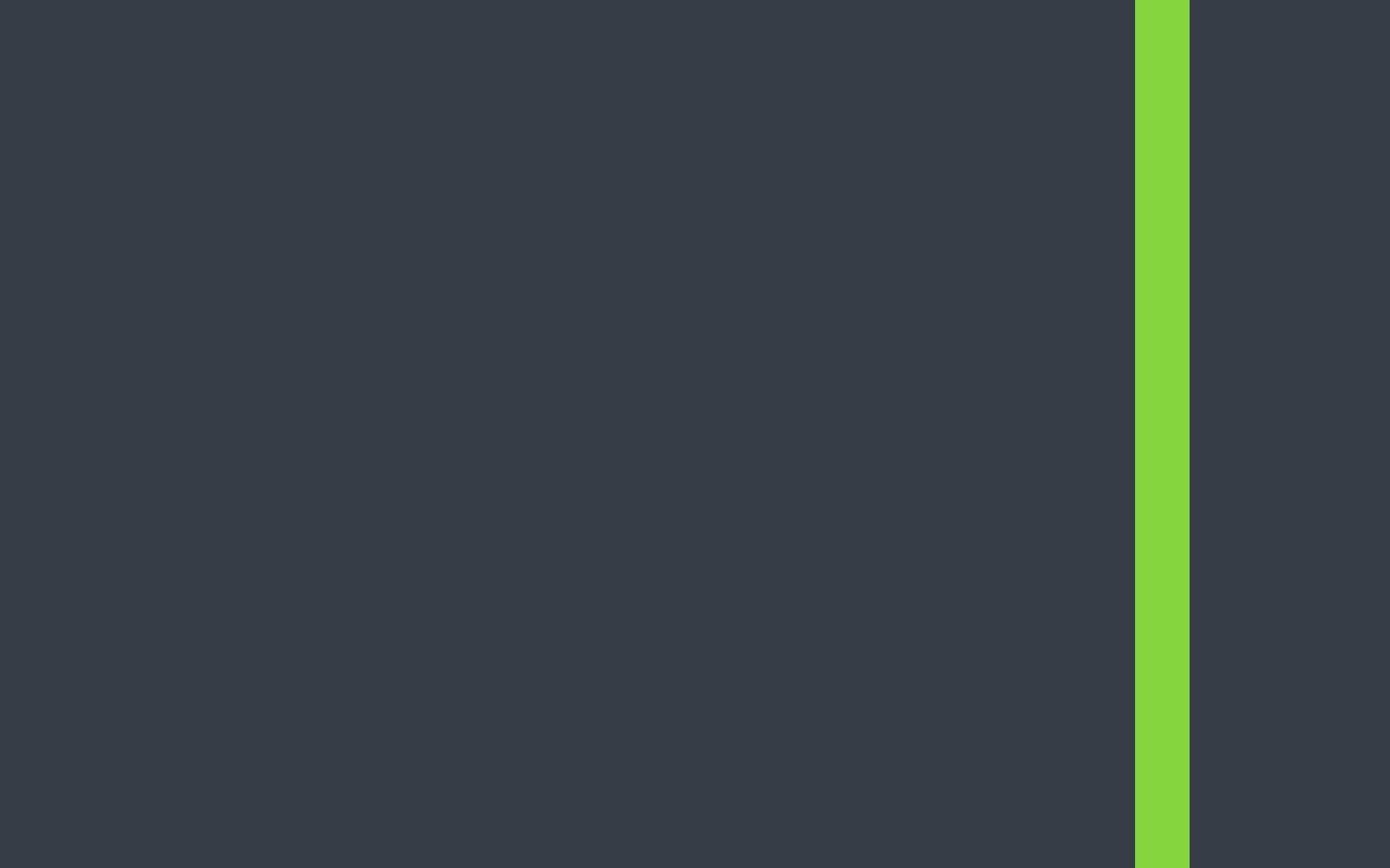




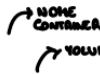
Sistemi Operativi Esame

Mattia Cosimi

January 2025

1 Prima Lezione: Introduzione alla riga di comando

- Creazione (e avvio) di un container con l'immagine del corso:

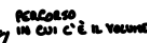
- `docker container run --name os2425` 
`--hostname sistemi-operativi -v dati:/corso -it chmod84/os2425`

- Avvio del container precedentemente creato ed avvio della shell

- `docker container start os2425`
 - `docker container exec -it os2425 /bin/bash`

- Aggiornamento dell'immagine del container:

- `docker container stop os2425` 

 - `docker container rm os2425`
 - `docker image pull`  `os2425`
 - `docker container run --name os2425` 
`--hostname sistemi-operativi -v dati:/corso -it chmod84/os2425` 


1.1 Significato di quello che scriviamo/vediamo

Se scriviamo **Docker** e premiamo invio mi fa vedere le varie azioni che posso fare, anche se eseguiamo **Docker container** ci da ulteriori suggerimenti. In **--name** possiamo mettere quello che vogliamo **--hostname** anche qui possiamo mettere quello che vogliamo sarebbe il nome della macchina, **-v** serve a specificare un nome di volume (**dati**), **:** dove vogliamo andare a rendere accessibile il volume di nome **dati**. **-it** serve a dire che il container che vogliamo creare deve essere "interattivo", a seguire il nome dell'immagine che ci voglio mettere dentro (una copia), avremmo potuto aggiungere **:nome versione**. Ora troveremo **corso@sistemi-operativi:~** si chiama prompt. A sinistra della chiocciola c'è il **nome utente** che sta utilizzando il sistema (ogni utente ha permessi diversi). Invece **sistemi-operativi** è l'**hostname**. **Tilde** indica la cartella in cui ci troviamo in questo momento (percorso **home directory**), **Dollaro** è un indicatore che ci dice che dopo dobbiamo inserire i comandi (**Dollaro** utente, **Cancelletto** amministratore).

1.2 Comandi

PWD (Print Working Directory) stampa la cartella nella quale stiamo lavorando (working directory).

Quando lavoriamo con le cartelle, il percorso può avere uno **slash** finale se siamo di fronte a una cartella, se siamo di fronte a un **file** non lo avrà.

CD (Change Directory) mi permette di cambiare cartella, prende come argomento **il percorso dove vogliamo andare**. Se non gli diamo argomenti ci riporta nella **home directory** (directory di casa: possiamo fare tutto quello che vogliamo).

Esistono due percorsi, se iniziano con / allora sono **assoluti** sennò sono **relativi**. Gli **assoluti** identificano tutto il percorso da fare a partire dalla radice, i **relativi** vengono utilizzati da Linux concatenando la wd con il percorso relativo. **Importanti:** directory . identifica la directory corrente, la directory .. rappresenta la directory superiore. Ovviamente si possono concatenare, es. ../../ ti porta indietro di due cartelle.

MKDIR (Make Directory) permette la creazione di una o più directory. L'argomento è un **percorso** che può essere relativo o assoluto. Per creare cartelle una dentro l'altra aggiungiamo **-p** (es. mkdir -p dir4/dir5/dir6)

LS (List) ci permette di vedere gli elementi della directory corrente. Si può inserire un argomento in modo tale da vedere il contenuto di quella directory.

2 Seconda Lezione: Introduzione alla riga di comando

2.1 Gestione degli spazi

Ci sono diversi modi: **utilizzo di " o ' (es. "folder 1")**. Oppure l'utilizzo del carattere di escape (es. folder(backslash)3).

2.2 Comandi (Gestione di un file)

Si può redirigere l'output di un comando su file tramite l'operatore > (es. ls > output.txt). **ATTENZIONE:** quest'operatore va a troncare il contenuto del file e riscriverlo, quindi se lo eseguo due volte il contenuto del file non si duplica.

Con l'operatore di ridirezione >> questo problema si risolve

CAT (Concatenate) (copio il contenuto del file all'interno dello stdout) Questo comando oltre a stampare a video il contenuto del file, prende come argomenti i **file** che voglio stampare a video ("li concatena").

ECHO stampa sullo standard output quello che gli passo da riga di comando. (es. echo "Ciao sono Stefano" > stefano.txt, ora il file stefano.txt verrà creato e avrà all'interno Ciao sono stefano).

LESS ci permette di vedere a schermo tutto il contenuto del file e scorrere con le frecce.

MORE ci permette di fare la stessa cosa di LESS "depotenziata". Per uscire dai comandi basta premere **Q**.

MV (Move) ci permette di spostare elementi all'interno del file system. (es. mv nomefile nomecartella/nomepercorso). Posso spostare più elementi basta che l'ultimo elemento della riga sia una cartella. Possiamo usarlo per **ridenominare** (es. mv combined.txt combined2.txt)

CP (Copy) ci permette di copiare un file. (es. cp nomefile percorso) è inoltre possibile copiare il contenuto dentro un altro file mettendo cp nomefile nome-file. (es. cp dir2/combined.txt ./combined-copia.txt crea nella directory corrente combined-copia.txt e metti dentro il contenuto di combined.txt)

RM (remove) ci consente di eliminare i file (es. `rm percorso/nomefile.txt`).

RMDIR (Remove diretory) ci consente di eliminare le cartelle. Non funziona se la cartella ha qualcosa all'interno.

RM ha una versione ricorsiva, che va a togliere tutto il contenuto di una cartella. **RM -r** . Posso aggiungere **-v** per un livello di verbosità maggiore ovvero ci fa capire meglio cosa accade (in questo caso ci fa capire cosa va a cancellare).

3 Terza Lezione: Introduzione alla riga di comando

3.1 Caratteri speciali

Il carattere **speciale** `*` serve a dire "sostituisci con zero o più caratteri qualunque. (Es: `qualcosa*` corrisponde a tutte le stringhe che hanno qualcosa come prefisso) (Es: `*qualcosa` corrisponde a tutte le stringhe che hanno qualcosa come suffisso) (Es: `*qualcosa*` corrisponde a tutte le stringhe che hanno al loro interno qualcosa)

Il carattere **speciale** `?` vuol dire sostituisci un carattere solo. (Es. guarda `* solo` che riempie con un solo carattere). **Entrambi** i caratteri non devono per forza comparire all'estremità della stringa.

3.2 Comandi

WC (word count). **WC -l** Ci permette di contare le righe all'interno di un file. **WC -c** per contare il numero di caratteri. **WC** normale ci dice `numeroRighe numeroDiParole numeroDiCaratteri`. Se inserisco il comando **senza** nome di file, il sistema si aspetta un input da **stdin**.

In questa lezione inoltre capiamo come mandare in input lo stream dello stdout.

Operatore di Pipe `|` ci consente di staccare stdout di un comando e mandarlo in input a un altro comando (es. `ls | wc`).

UNIQ ci consente di contare il numero di righe uniche all'interno di (le righe identiche devono essere adiacenti) un file. Un esempio potrebbe essere: `cat combined.txt | uniq | wc -l` (stiamo chiedendo di mandare in input al comando `uniq` il cat di `combined.txt` che a sua volta manderà in input il numero di righe differenti a `wc -l`).

SORT ordina alfabeticamente il contenuto del file.

4 Quarta lezione: Introduzione ai sistemi operativi

Le **chiamate di sistema** hanno l'aspetto di funzioni che però non fanno parte di una libreria ma bensì sono direttamente esposte dal nucleo del sistema operativo.

4.1 Programma cpu.c

```
1      #include <stdio.h>
2      #include <stdlib.h>
3      #include <sys/time.h>
4      #include <assert.h>
5      #include "common.h"
6
7      int
8      main(int argc, char *argv[])
9      {
10         if (argc != 2) {
11             fprintf(stderr, "usage: cpu <string>\n");
12             exit(1);
13         }
14         char *str = argv[1];
15         while (1) {
16             Spin(1); // Repeatedly checks the time and
17                     // returns once it has run for a second
18             printf("%s\n", str);
19         }
20         return 0;
21     }
```

Annotations:

- Line 9: *CONTINUA CHE NON VENGA INSERITO NIENTE DI INSOLITO*
- Line 15: *FUNZIONE CHE FA ASPETTARE UN SECONDO*
- Line 19: *STAMPA IL CARATTERE INSERITO*

Simple Example(cpu.c): Code That Loops and Prints

Si trova dentro ostep-code. Controlla se gli argomenti sono diversi da 2, in caso stampa una riga che dice guarda l'uso è una stringa. Dopodichè ogni secondo stampa una stringa. **ATTENZIONE:** non lasciare i file dentro ostep-code, vengono cancellati, lasciali dentro /home/corso.

gcc -Wall -o cpu cpu.c per la compilazione di un file (-Wall è opzionale).

Su windows se inserisco un nome di un file per eseguirlo lui di default lo va a cercare nella **working directory**, su linux non c'è quindi bisogna specificare **./cpu**

TOP ci consente di vedere varie informazioni, tra cui i processi (equivalente del task manager).

& ci consente di mandare in background il comando che la precede e di mandare in esecuzione il comando che la succede. Per cancellare tutte le istanze di programmi **killall cpu**.

4.2 Programma mem.c

```
1      #include <unistd.h>
2      #include <stdio.h>
3      #include <stdlib.h>
4      #include "common.h"
5
6      int
7      main(int argc, char *argv[])
8      {
9          int *p = malloc(sizeof(int)); // a1: allocate some
                                         memory
10         assert(p != NULL); → UN IF CHE SE NON ASPETTA LA COMEZIONE TERMINAL PROGRAM
11         printf("(d) address of p: %08x\n",
12                getpid(), (unsigned) p); // a2: print out the
                                         address of the memmory
13         *p = 0; // a3: put zero into the first slot of the memory
14         while (1) {
15             Spin(1);
16             *p = *p + 1;
17             printf("(d) p: %d\n", getpid(), *p); // a4
18         }
19         return 0;
20     }
```

P è un puntatore a intero. **Assert** è un'istruzione in cui se quello che c'è tra parentesi non è soddisfatto il programma viene terminato. La prima **printf** dice: tra parentesi c'è un numero intero che viene sostituito dal **processId** (getPid()). Dopo address di p viene messo in **esadecimale** l'indirizzo del puntatore. Quindi ogni secondo quest'applicazione **aggiorna e incrementa** di 1 il valore di p. Sulla cartella ostep-code c'è una versione leggermente diversa con la funzione **atoi** che ci converte un numero scritto a caratteri e ci restituisce un intero.

```
void *worker(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        counter++;
    }
    return NULL;
}

int main(int argc, char *argv[]) {
    if (argc != 2) {
        fprintf(stderr, "usage: threads <loops>\n");
        exit(1);
    }
    loops = atoi(argv[1]);
    pthread_t p1, p2;
    printf("Initial value : %d\n", counter);
    Pthread_create(&p1, NULL, worker, NULL);
    Pthread_create(&p2, NULL, worker, NULL);
    Pthread_join(p1, NULL);
    Pthread_join(p2, NULL);
    printf("Final value : %d\n", counter);
}
```

Ora viene analizzata **threads.c**. Inizialmente chiama una variabile **counter** e la imposta a 0 (non presente nell'immagine) e una variabile globale **loops**. Oltre al **main** abbiamo un'altra funzione chiamata **worker**, la quale incrementa **counter** **loops** volte di uno. Per quanto riguarda il **main**, di nuovo vi è l'utilizzo dei **thread**. Immaginiamo che fino ad oggi abbiamo scritto le applicazioni con un unico filo di esecuzione (può utilizzare al più un core alla volta), vediamo ora un'applicazione con più fili e che quindi può utilizzare più core contemporaneamente.

pthread_t dichiaro due thread e li creo con *pthread_create*, i quali andranno a eseguire la funzione chiamata **worker**. Quindi attendiamo che il primo thread termini, attendiamo che il secondo termini e poi andiamo a stampare il valore di **counter**.

4.2.1 Problema di Thread.c

Ci aspettiamo che il valore di **counter** venga raddoppiato rispetto al numero di **loops** (essendo due i thread), questo però non funziona per valori molto grandi, quello che noi pensiamo essere un'istruzione su una riga (**counter++**), in realtà richiede che: **counter=** significa andare a scrivere qualcosa dentro **counter**, 1 istruzione, a destra c'è **counter+1** che significa, accedi alla memoria leggi il valore di **counter** incrementalo e scrivi dentro **counter** il risultato. Le tre istruzioni non vengono eseguite in modo **atomico**. Cosa succede? i thread cercano contemporaneamente di aggiornare **counter**, il sistema operativo che assegna una porzione di tempo per ogni cpu, interrompe un processo ne fa continuare un'altro ecc... (es. il primo thread legge il valore di **counter**, vede che è 10, in questo momento il sistema operativo dice ok non è più il tuo turno è il turno dell'altro thread, il quale legge il valore di thread (10) somma uno e lo rimette in **counter** quindi **counter** ora è 11, continuo finché non mi interrompe il sistema operativo, magari sono arrivato a 450, ora il sistema operativo ridà il controllo al primo thread il quale aveva appena letto 10 ora va a fare +1 e quindi **counter** sarà uguale a 11. ecco perché il valore di **counter** sarà minore della somma dei loop). Per valori molto bassi funziona perché i thread hanno lavorato in modo **sequenziale**.

4.3 Programma io.c (Persistenza)

```
1      #include <stdio.h>
2      #include <unistd.h>
3      #include <assert.h>
4      #include <fcntl.h>
5      #include <sys/types.h>
6
7      int
8      main(int argc, char *argv[])
9      {
10         int fd = open("/tmp/file", O_WRONLY | O_CREAT
11                    ↑ 1 1 1
12                    1 1 1 O_TRUNC, S_IRWXU);
13         assert(fd > -1);
14         int rc = write(fd, "hello world\n", 13);
15         assert(rc == 13);
16         close(fd);
17         return 0;
18     }
```

Questo programma tramite la chiamata di sistema **open** va a aprire un file che si trova nel percorso **/tmp/file**. Per quanto riguarda i parametri a sinistra della **open**, sono dei "permessi", **wonly** (apertura del file di sola scrittura), **creat** (se il file che ti ho specificato nel percorso non esiste crealo), **trunc** (se il file dovesse esistere troncalo), la **assert** ha lo stesso scopo di prima. La chiamata **write** scrive nel file specificato, il tredici alla fine consiste nella lunghezza della stringa **hello world** (n. byte). La **write** ci ritorna il numero di caratteri scritti nel file (13).

5 Quinta lezione: process API

5.1 Scrittura di un file (editor)

Quando voglio andare a scrivere un file ho due modi: il primo crearlo all'interno del container utilizzando un editor di testo **"da riga di comando"** oppure utilizzare un editor di testo **qualsiasi** e copiare il file all'interno del container per andarlo a compilare.

NANO (nano nomefiledacreare.c) è l'editor di testo da riga di comando. Ci illumina la sintassi. Per **salvare** ctrl+O - invio. Per **uscire** ctrl+X.

Per il secondo metodo, apriamo l'editor di testo e scriviamo il codice. Per **portarlo all'interno del container** scriviamo **docker cp** (nomeFile) nomeContainer:/percorso.

Il professore suggerisce di copiare la cartella ostep-code in /home/corso in modo tale da non perderne il contenuto quando andiamo a aggiornare l'immagine. **cp -r nomeCartella Percorso**.

5.2 Chiamata di sistema: Fork()

p1.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(int argc, char *argv[]){
    printf("hello world (pid:%d)\n", (int) getpid());
    int rc = fork();
    if (rc < 0) {    // fork failed; exit
        fprintf(stderr, "fork failed\n");
        exit(1);
    } else if (rc == 0) { // child (new process)
        printf("hello, I am child (pid:%d)\n", (int) getpid());
    } else {          // parent goes down this path (main)
        printf("hello, I am parent of %d (pid:%d)\n",
            rc, (int) getpid());
    }
    return 0;
}
```

La **fork()** essendo una chiamata di sistema, è considerata un **servizio del kernel**. Ci consente di **sdoppiare** il processo che lo chiama, va a creare un nuovo processo che sarà un figlio del processo chiamante. La **fork** ritorna 0 se l'esecuzione continua sul processo figlio, ritorna **il pid del processo figlio** se l'esecuzione continua sul processo padre. Ritorna un valore **minore** di 0 se la fork fallisce.

MAN, fornisce una documentazione completa di qualsiasi chiamata di sistema. (es. man chiamataDiSistema)

5.3 Chiamata di sistema: Wait()

Attende il cambiamento di stato di un processo (a noi interessa la terminazione di un processo).

p2.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(int argc, char *argv[]){
    printf("hello world (pid:%d)\n", (int) getpid());
    int rc = fork();
    if (rc < 0) {    // fork failed; exit
        fprintf(stderr, "fork failed\n");
        exit(1);
    } else if (rc == 0) { // child (new process)
        printf("hello, I am child (pid:%d)\n", (int) getpid());
    } else {           // parent goes down this path (main)
        int wc = wait(NULL);
        printf("hello, I am parent of %d (wc:%d) (pid:%d)\n",
            rc, wc, (int) getpid());
    }
    return 0;
}
```

L'unica differenza con il programma di prima è quando l'esecuzione viene continuata dal processo padre. C'è una **WAIT** la quale ci aspetteremo che conterrà il valore del **processo che ha cambiato stato** (processo figlio). La vera **differenza** è che questa volta, il processo padre non potrà mai stampare la riga di testo se prima non muore il processo figlio (**WAIT**).

Se utilizziamo **WAIT PID** dobbiamo accertarci come processo padre di salvare da qualche parte tutti i processi figli che generiamo. Altra nota, le chiamate di sistema si trovano al capitolo 2 del MAN, quindi in caso ci fossero comandi con lo stesso nome della chiamata di sistema, dovremmo specificare il capitolo (man 2 write).

5.4 Chiamata di sistema: Open(), Write(), Close()

Open() ci serve per aprire un file, **Write()** ci serve per scrivere qualcosa all'interno di un file, **Close()** ci serve per chiudere il file precedentemente aperto.

```
1 #include <stdio.h>
2 #include <fcntl.h> // Da includere per utilizzare open
3 #include <string.h> //Da includere per utilizzare strlen
4 #include <unistd.h> //Da includere per utilizzare write
5
6 int main(){
7
8     char messaggio[] = "Ciao!";
9     int fd = open("pippo.txt", O_CREAT | O_TRUNC | O_WRONLY , S_IRWXU);
10    // fd sta per file descriptor
11    write(fd, messaggio, strlen(messaggio));
12    close(fd);
13 }
```

Importante: Quando utilizziamo la **open** dobbiamo sempre specificare se vogliamo effettuare lettura (**RDONLY**), scrittura (**WRONLY**) o entrambe (**RDWR**). Il **file descriptor** è un numero intero ed è un riferimento al file che è stato aperto, infatti tutte le interazioni future verranno fatte utilizzando il riferimento del file descriptor.

5.4.1 Write()

Quello che fa la **write** è prendere un puntatore all'inizio della memoria di messaggio (**secondo parametro** indirizzo di base della memoria dal quale devo iniziare a leggere i dati da scrivere da lì vado avanti per quanto indicato dal

terzo parametro) e va avanti per la lunghezza specificata dal terzo parametro. La write **ritorna** il numero di byte che sono stati scritti dalla chiamata stessa.

6 Sesta lezione: process API

Continuiamo a vedere **read** e **write**. **S_IRWXU** ci dice i permessi che ha il file sul file system.

```
1 #include <stdio.h>
2 #include <fcntl.h> // Da includere per utilizzare open
3 #include <string.h> //Da includere per utilizzare strlen
4 #include <unistd.h> //Da includere per utilizzare write
5
6 int main(){
7
8     char messaggio[] = "Ciao!";
9     int buffer = 4;
10    int fd = open("pippo.txt", O_CREAT | O_TRUNC | O_WRONLY , S_IRWXU);
11    // fd sta per file descriptor
12    printf("fd: %d\n", fd);
13    int bytes = write(fd, &buffer, sizeof(int)); //in questo modo nel file pippo
14    // verranno messi 4 caratteri "spazzatura"
15    printf("bytes: %d\n", bytes);
16    close(fd);
17 }
```

Ora ci occupiamo della lettura.

```
1 #include <stdio.h>
2 #include <fcntl.h> // Da includere per utilizzare open
3 #include <unistd.h> //Da includere per utilizzare read
4
5 int main(){
6
7     int fd = open("pippo.txt", ORDONLY , S_IRWXU);
8     int buffer = 0;
9     read(fd, &buffer, sizeof(int));
10    printf("Buffer: %d\n", buffer);
11    close(fd);
12 }
```

6.1 Chiamata di sistema: Exec()

La chiamata di sistema **Exec()** ci permette di eseguire altre applicazioni da applicazioni che noi stiamo scrivendo (andandone a richiamare l'eseguibile).

```
1 #include <stdio.h>
2 #include <fcntl.h> // Da includere per utilizzare open
3 #include <unistd.h> //Da includere per utilizzare read
4 #include <string.h> //Da includere per strdup
5
6 int main(){
7
8     int fd = open("pippo.txt", ORDONLY , S_IRWXU);
9     int buffer = 0;
10    read(fd, &buffer, sizeof(int));
```

```

11     printf(" Buffer : %d\n" , buffer );
12     close( fd );
13
14     char *myarg[2]; //array di argomenti per la exec
15                     // Percorso dell'eseguibile
16                     // Argomenti da passare all'eseguibile
17                     // Null ultimo argomento
18     myarg[0] = strdup("ls");
19     myarg[1] = NULL;
20     execvp(myarg[0] , myarg);
21
22 }

```

Strdup si prende una stringa come parametro, fa una malloc per questa stringa ci mette dentro il contenuto della memoria che ha appena allocato e restituisce il puntatore a quell'area di memoria. Per quest'esempio utilizziamo **execvp**, la **famiglia v** mi permette di passare come parametri l'eseguibile nel formato array di stringhe, la **famiglia p** fa una ricerca all'interno della variabile d'ambiente path dell'eseguibile che voglio eseguire esattamente come se fossi all'interno della shell. Tutto ciò che c'è **sotto la exec** non verrà eseguito per questo motivo prima della exec si chiama una fork che chiamerà un altro processo a eseguire altri comandi.

7 Settima lezione: introduzione programmazione concorrente

Si va a svelare la soluzione del 4 esercizio della seconda esercitazione in laboratorio. (slide su my malloc).

7.1 Testo dell'esercizio

Esercizio 4: Implementazione di my_malloc() e my_free()

Scrivi un programma che implementi le funzioni *my_malloc()* e *my_free()*. Queste funzioni devono comportarsi rispettivamente come le funzioni *malloc()* e *free()*, cioè: la prima deve ritornare un puntatore void ad una area di memoria utilizzabile della dimensione specificata come parametro; la seconda deve liberare l'area di memoria alla quale fa riferimento il puntatore passato come parametro. L'intero programma deve essere realizzato senza utilizzare le funzioni *malloc()* e *free()*. E' possibile utilizzare una sola volta *mmap()* e *munmap()*, per le quali sono disponibili le pagine di man.

Quindi la funzione my malloc, dovrà prendere un argomento (dimensione) e ritornare un void*. La my free quindi dovrà prendere soltanto un puntatore e tornare un intero che ci faccia capire se è andata a buon fine o meno.

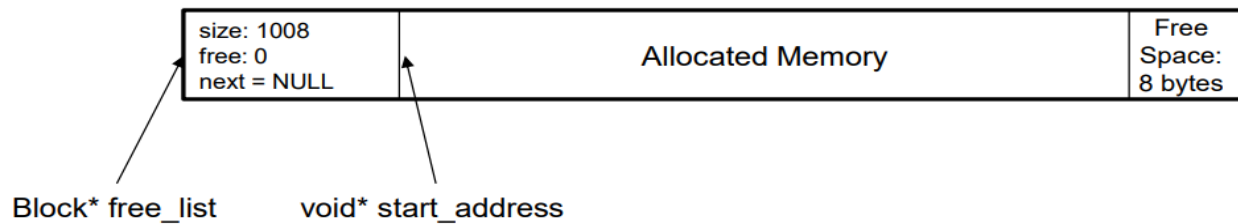
MMAP è una funzione che possiamo utilizzare in due modi diversi: il **primo** modo è quello di simulare il comportamento della malloc (infatti prende più argomenti che in realtà la malloc mette di default es. permessi di memoria all'area di memoria). Il **secondo** è quello di creare una mappa con nome: un riferimento in memoria rispetto a un file che si trova nel file system. Potremmo avere anche mappe anonime (quello che corrisponde a una malloc) le quali sono porzioni di memoria che non hanno un corrispettivo file nel file system. Il **primo** parametro della mmap è un'indirizzo di memoria che noi vorremmo come base di partenza per la memoria che vogliamo allocare (si può non specificare). **Secondo** parametro dimensione dell'area di memoria che vogliamo allocare. **Prot** indica i permessi. **Flags** lo utilizziamo per map shared e map private. Il **quinto** parametro è un fd e per finire abbiamo un offset (che serve a dire da dove partire).

7.2 Idea della risoluzione

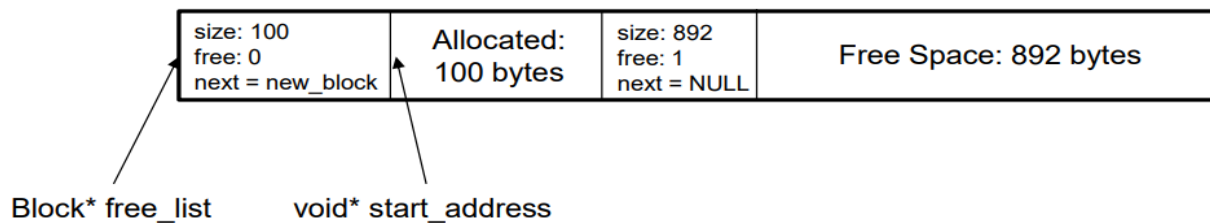
Torniamo alla risoluzione dell'esercizio, l'idea è: uso mmap una volta per andare a fare l'allocazione di memoria, quest'area sarà tutta e sola l'area di memoria che il nostro computer andrà a utilizzare, infatti tutte le successive

allocazioni con l'utilizzo della my malloc verranno erogate a partire da questa area di memoria che ho inizializzato con la mmap. Ho bisogno di scrivere da qualche parte quanto sto allocando in modo tale che la my free sappia quanto deve deallocare. A tal proposito uso una struttura dati che chiamo **Block*** che mi va a contenere una dimensione, se il blocco di memoria è libero e qual'è il prossimo blocco di memoria.

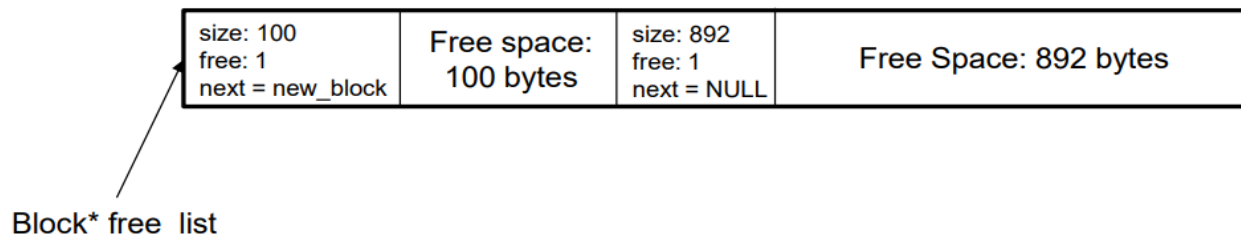
Es. se volessi fare una my malloc di 1000 potrei farlo se decido di allocare ad esempio (1008 byte) ma mi rimarrebbero solo 8 byte per contenere i metadati con le informazioni sui blocchi di memoria.



La my free va indietro di 16 byte rispetto a start address legge il blocco e sa quanta memoria deve liberare. La libera semplicemente settando a 1 la variabile free. Vediamo ora una my malloc da 100 byte.



Quell' 892 è dato da $1024 - 16 - 100 - 16$.



Ora posso liberare la memoria cambiando semplicemente il parametro free. Il problema è che la memoria non essendo contigua posso utilizzare semplicemente solamente 992 byte (problema della frammentazione interna). La soluzione è quella di fare un merge di blocchi liberi adiacenti.

size: 100 free: 1 next = ...	Free space: 100 bytes	size: 123 free: 0 next = ...	Allocated: 123 bytes	size: 753 free: 1 next = ...	Free Space: 753 bytes
------------------------------------	--------------------------	------------------------------------	-------------------------	------------------------------------	-----------------------

Se avessimo una situazione del genere, non potremmo risolverla.

7.3 Soluzione in C

Abbiamo due macro una per la dimensione della memoria che vogliamo utilizzare e una per definire la dimensione minima del blocco che posso creare. La struct del blocco è fatta esattamente come viene specificato nelle slide (size_t size, int free e struct Block* next).

7.3.1 init_memory_pool()

```
void init_memory_pool() {
    if (memory_pool == NULL) {
        memory_pool = (Block*) mmap(NULL, MEMORY_POOL_SIZE, PROT_READ | PROT_WRITE, MAP_PRIVATE, -1, 0);
        if (memory_pool == MAP_FAILED) {
            perror("mmap");
            exit(1);
        }

        memory_pool->size = MEMORY_POOL_SIZE - sizeof(Block);
        memory_pool->free = 1;
        memory_pool->next = NULL;
        free_list = memory_pool;
    }
}
```

Per prima cosa vado a inizializzare il **memory pool** (mettere il pezzettino di memoria all'inizio che mi dice la memoria utilizzabile). Se **memory pool** è nullo allora lo vado a creare con **mmap** (parametri: NULL(non ci interessa suggerire l'indirizzo da cui partire), memory pool size (dimensione che vogliamo allocare), i prot che inseriamo sono read e write dato che vogliamo leggerla e scriverla, dopodichè diciamo che la mappa deve essere privata e anonima essendo anonima fd=-1 e l offset non è specificato). Le istruzioni dopo sono in linea col funzionamento spiegato prima (imposto i parametri).

7.3.2 split_block

```
void split_block(Block* block, size_t size) {
    Block* new_block = (Block*)((char*)block + sizeof(Block) + size);
    new_block->size = block->size - size - sizeof(Block);
    new_block->free = 1;
    new_block->next = block->next;

    block->size = size;
    block->free = 0;
    block->next = new_block;
}
```

Richiesta di allocazione che permette successive richieste di allocazione. Creo un puntatore di base al nuovo blocco creato a partire da: il blocco corrente (quello che voglio splittare) + dimensione della struttura dati del blocco + dimensione dei dati allocati per quel blocco. La dimensione del nuovo blocco che andiamo a creare è quindi la dimensione del blocco che ho a disposizione - la dimensione dei dati che stiamo andando a utilizzare - dimensione del blocco che ho creato.

7.3.3 print_blocks

```
void print_blocks(Block* block) {
    int i = 0;
    while (block != NULL) {
        printf("Blocco %d:\n", i++);
        printf("\tSize: %lu\n", block->size);
        printf("\tFree: %d\n", block->free);
        block = block->next;
    }
    printf("-----\n");
}
```

7.3.4 my_malloc

```
void* my_malloc(size_t size) {
    if (size <= 0) {
        return NULL;
    }

    init_memory_pool();

    Block* current = free_list;
    while (current != NULL) {
        if (current->free && current->size >= size) {
            if (current->size > size + MIN_BLOCK_SIZE) { //Can I create a
                split_block(current, size);
            } else {
                current->free = 0;
            }
            printf("Memoria allocata, nuovo layout:\n");
        }
    }
}
```

Quindi la nostra my malloc ritorna e prende i parametri precedentemente citati, quindi sostanzialmente partiamo dall'inizio dei blocchi fino a quando non troviamo un blocco di dimensione sufficiente alla richiesta che è stata fatta. Quindi se entra un'altro blocco andiamo a richiamare split block sennò dichiariamo tutta la memoria restante occupata. Andiamo quindi a ritornare l'indirizzo di base della memoria allocata.

```
        print_blocks(free_list);
        return (void*)((char*)current + sizeof(Block));
    }
    current = current->next;
}

return NULL; // Out of memory
}
```

7.3.5 merge_blocks

```
void merge_blocks(Block* block) {
    while (block->next != NULL && block->next->free) {
        block->size += sizeof(Block) + block->next->size;
        block->next = block->next->next;
    }
}
```

7.3.6 my_free

```
void my_free(void* ptr) {
    if (ptr == NULL) {
        return;
    }

    Block* block = (Block*)((char*)ptr - sizeof(Block));
    block->free = 1;
    merge_blocks(block);
    printf("Memoria liberata, nuovo layout:\n");
    print_blocks(free_list);
}
```

Prendo il puntatore passato e ci sottraggo la dimensione dei metadati in modo tale da liberare la memoria.

7.3.7 cleanup_memory_pool

```
void cleanup_memory_pool() {
    if (memory_pool != NULL) {
        if (munmap(memory_pool, MEMORY_POOL_SIZE) == -1) {
            perror("munmap");
        }
        memory_pool = NULL;
        free_list = NULL;
    }
}
```

7.3.8 Main

```
int main() {
    char* ptr1 = (char*) my_malloc(100);
    int *array = my_malloc(1000 * sizeof(int));

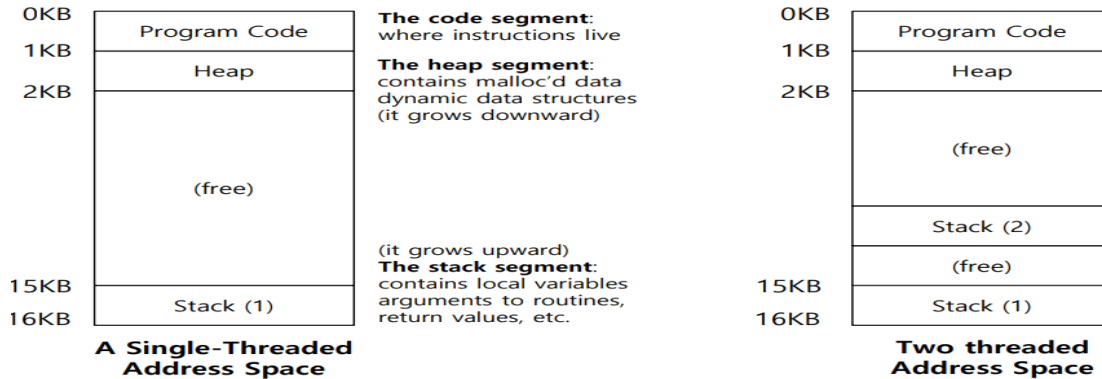
    my_free(array);
    my_free(ptr1);

    cleanup_memory_pool();

    return 0;
}
```

7.4 Programmazione Concorrente

Gestione degli interrupt. Ogni processo ha un **pc (program counter)**. In applicazioni multi thread si possono avere più **program counter**. Una differenza sostanziale è che tutti i thread si appoggiano allo stesso spazio degli indirizzi, cosa non possibile nella programmazione multi processo. Ogni thread ha quindi il proprio insieme di **registri**. Con i thread si fa un context switch più leggero e i loro dati vengono salvati in memoria in un **Thread Control Blocks (TCBs)**. Come cambia lo spazio degli indirizzi:



La programmazione multi thread va a migliorare l'efficienza.

7.4.1 Primo esempio sulla programmazione multi thread

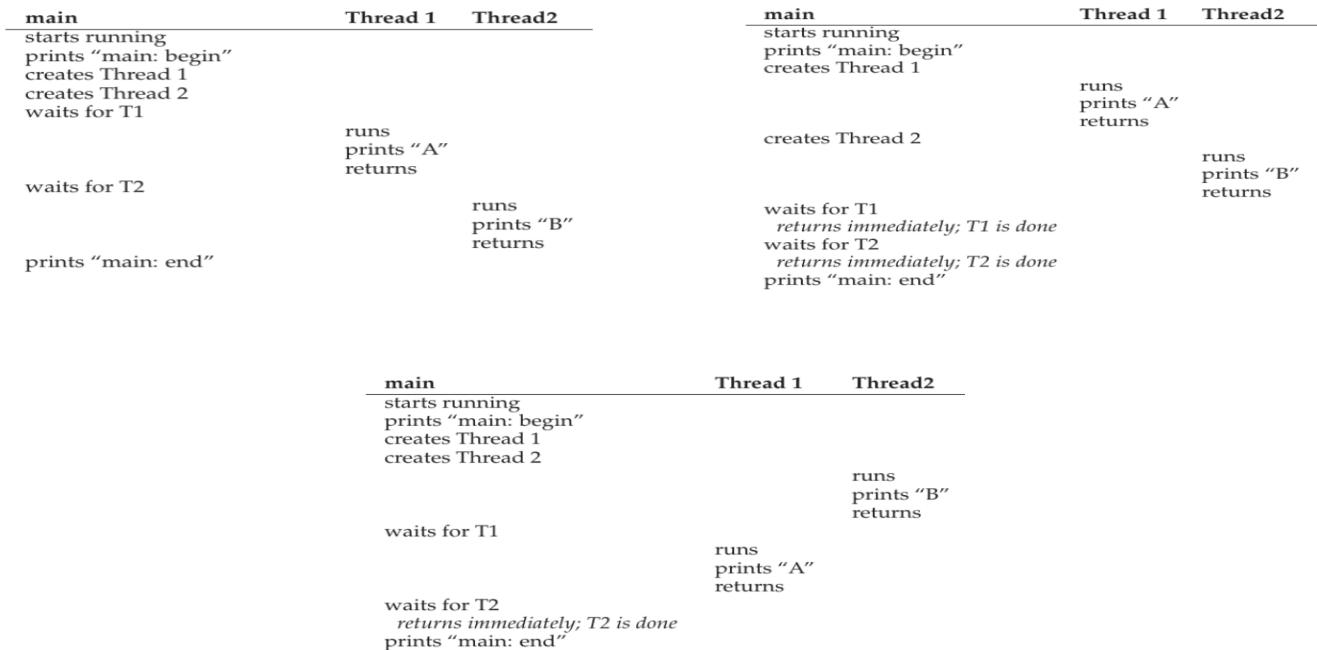
```
#include <stdio.h>
#include <assert.h>
#include <pthread.h>

void *mythread(void *arg) {
    printf("%s\n", (char *) arg);
    return NULL;
}

int
main(int argc, char *argv[]) {
    pthread_t p1, p2;
    int rc;
    printf("main: begin\n");
    rc = pthread_create(&p1, NULL, mythread, "A"); assert(rc == 0);
    rc = pthread_create(&p2, NULL, mythread, "B"); assert(rc == 0);
    // join waits for the threads to finish
    rc = pthread_join(p1, NULL); assert(rc == 0);
    rc = pthread_join(p2, NULL); assert(rc == 0);
    printf("main: end\n");
    return 0;
}
```

Utilizzeremo applicazioni che fanno uso di **Pthread** (libreria più utilizzata per programmazione concorrente in c). Le prime due variabili di tipo **Pthread_t** sono degli identificatori ai thread che sto per andare a creare/eseguire. Successivamente utilizziamo **pthread_create** è la funzione che fa sì che un thread possa nascere e possa essere eseguito. Gli passiamo come **argomenti**: un riferimento a oggetto di tipo pthread_t che può andare a modificare, il terzo argomento è un puntatore a una funzione, l'ultimo parametro è un argomento da passare alla funzione. Quindi quando questa funzione viene eseguita, quello che succede è che oltre al thread principale verranno creati due nuovi thread con i propri pc. **Pthread_join** il thread che va a eseguire quella funzione si mette in attesa del thread identificativo p1 (il thread principale non andrà avanti fino a che p1 non avrà terminato la sua esecuzione).

7.4.2 Possibili scenari di esecuzione



Esecuzioni che dipendono dallo scheduler. Il codice è nel container se vuoi eseguirlo devi togliere le include "" e mettere le p minuscole alle funzioni. Se voglio compilare qualcosa che fa uso di **pthread** devo scrivere **gcc -o t0 t0.c -pthread**

7.4.3 Pthread_create

```
int pthread_create(pthread_t *restrict thread,
                  const pthread_attr_t *restrict attr,
                  void *(*start routine)(void *),
                  void *restrict arg);
```

Il primo argomento è un puntatore a pthread t che sarà l'identificatore del thread che abbiamo creato. Poi c'è la variabile che definisce gli attributi specifici del thread che stiamo andando a creare. Il terzo parametro è il puntatore a funzione (indirizzo di memoria dove inizia quella funzione (quello che verrà messo dentro il pc del thread che stiamo andando a creare)). L'ultimo sono gli argomenti da passare al thread.

7.4.4 Pthread_join

```
int pthread_join(pthread_t thread, void **retval);
```

Il secondo parametro è un doppio puntatore a un valore tornato dal thread.

8 Ottava lezione: continuo su programmazione concorrente

Inizialmente ci mostra come migliora l'esecuzione sfruttando la programmazione concorrente.

8.1 Moltiplicazione tra matrici (senza thread)

Ci fa vedere le statistiche sui tempi di esecuzione con la funzione `gettimeofday`, non utilizziamo una semplice `time` perchè lavora a una grana troppo grossa e vogliamo qualcosa di più **preciso**. Questa funzione si appoggia a una struct di tipo **timeval**: una struttura composta di due campi uno per i secondi e uno per i microsecondi. Come primo parametro prende un riferimento a variabili di tipo **timeval** (es. `start`) in modo tale che prenda secondi e microsecondi.

8.2 Moltiplicazione tra matrici (con thread)

Ogni thread lavorerà su un sott'insieme di righe della matrice, l'esecuzione è migliore. Ci fa vedere poi il file che si trova nella home `.wslconfig`. Se lo apro vedo il numero di processori impostati, per cambiarli, li cambio sul file, stoppo il container, digito `wsl --shutdown` che va a riavviare la macchina virtuale sottostante ai container, applicando l'impostazione.

8.3 Produttore-Consumatore

Quest'esercizio riprende un tipo di **architettura software** su cui molto spesso ci si va a lavorare. Funzione con almeno due agenti uno che **produce** qualche tipo di informazione, uno che **consuma** l'informazione. I due condividono una struttura dati per comunicare, il produttore mette all'interno di una **pila** l'informazione, il consumatore la preleva e ci lavora sopra. Questo problema è di **sincronizzazione** tra i due thread (agenti). Ci suggerisce l'utilizzo di `random` e `usleep` (diversifica da `sleep` perchè ci fa attendere micro secondi la u sta per μ).

```
1 #include <stdio.h>
2 #include <stdlib.h> // per la calloc e la random
3 #include <unistd.h> // per la usleep
4 #include <pthread.h> // per i thread
5
6 #define SIZE 10
7
8 struct stack{
9
10     int* array;
11     int capacita;
12     int occupati;
13
14 }stack;
15
16 stack pila;
17
18 pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER; // per sincronizzare i thread
19
20 void init(){
21
22     pila.array = (int*) calloc(SIZE, sizeof(int));
23     pila.capacita=SIZE;
24     pila.occupati=0;
25
26 }
27
28 void push(int v){
```

```

29
30     if (pila.occupati < pila.capacita){
31
32         pila.array[pila.occupati] = v;
33         pila.occupati++;
34     }
35 }
36
37 //int perche vogliamo che ci restituisca l'ultimo elemento che era stato inserito
38 int pop(){
39
40     int ret=-1;
41     if (pila.occupati > 0){
42
43         ret = pila.array[pila.occupati-1];
44         pila.occupati--;
45     }
46
47     return ret;
48 }
49
50 void* produttore(void* arg){
51
52     while (1){
53
54         // Attesa random
55         usleep(random()%1e6);
56         pthread_mutex_lock(&lock);
57         // Controllo se la pila e libera
58         if( pila.occupati < pila.capacita ){
59             // Elementi da inserire nella pila
60             int elementi = random() % (pila.capacita - pila.occupati);
61             for (int i = 0; i < elementi; i++){
62
63                 int v = random();
64                 push(random());
65                 printf("[Produttore]-Inserito %d\n", v);
66             }
67         }
68         pthread_mutex_unlock(&lock);
69     }
70 }
71
72
73 void* consumatore(void* arg){
74
75     while (1){
76
77         usleep(random()%(int)1e6);
78         pthread_mutex_lock(&lock);
79         if (pila.occupati > 0){
80
81             int elementi = random() % pila.occupati;
82             for ( int i=0; i < elementi ; i++ ){

```

```

83
84         int v = pop();
85         printf("[Consumatore] - Letto %d\n", v);
86     }
87 }
88 pthread_mutex_unlock(&lock);
89 }
90 }
91
92
93 int main(){
94
95     init();
96     pthread_t prod, cons;
97
98     pthread_create(&prod, NULL, produttore, NULL);
99     pthread_create(&cons, NULL, consumatore, NULL);
100
101     pthread_join(prod, NULL);
102     pthread_join(cons, NULL);
103 }

```

Per prima cosa creo la **struttura dati** con cui voglio lavorare. Creo una funzione che mi permetta di inizializzare la struttura dati. Quello che inizia con `define` è una macro. Ora creo **push** e **pop**. Dopo aver implementato tutto quello che serve per la struttura dati vado a implementare le funzioni **produttore** e **consumatore**. Random e $1e6$ (10^6) sono di due tipi diversi, random è considerato un long int $1e6$ un double, quindi bisogna fare un **cast** su $1e6$. Una volta implementati gli elementi da inserire nella pila, ci ricordiamo che dobbiamo attendere il consumatore. Infatti se lo spazio è pieno, devo soltanto attendere fino a che non mi si libera dello spazio. Nel consumatore, posso andare a **prelevare** soltanto se c'è qualcosa nella pila. Il **problema** sta nel fatto che dobbiamo sincronizzare la pila su cui avvengono push e pop e la variabile pila occupati. Il professore continua a consigliare fortemente l'utilizzo del `man`. Se lasciassimo l'implementazione con solamente `pthread create` il **thread principale** finirebbe prima degli altri due e quindi l'esecuzione terminerebbe. A tal proposito utilizziamo **pthread join** che fa sì che il thread principale aspetti l'esecuzione dei figli. A questo punto il programma non sarebbe del tutto corretto, vi sono da verificare le **race condition**. Infatti il programma sembra funzionare a causa del fatto che l'attesa fa sì che i thread non si sovrappongano ma in realtà potremmo sfociare in **segmentation fault**. Quindi se il **produttore** sta inserendo qualcosa all'interno della pila, il **consumatore** deve aspettare, se il **consumatore** sta rimuovendo dalla pila, il **produttore** deve aspettare. A tal proposito utilizzo **pthread mutex lock** e **unlock**.

9 Richiamo teorico sui Lock

9.1 Sezione critica e Race Condition

Sezione critica: è un pezzo di codice che è potenzialmente eseguito da più thread, che accedono a una variabile condivisa nello stesso momento. Più thread che eseguono una sezione critica possono risultare in

Race condition: il risultato della computazione dipende dal tempo di esecuzione del codice.

Per risolvere questo problema, abbiamo bisogno di un meccanismo flessibile, un meccanismo permetta di accedere a un thread alla volta alla sezione critica

9.2 Lock

Un **lock** è una variabile che contiene informazioni su quale thread può eseguire la sezione critica. L'idea di base è: la variabile lock mantiene lo stato del lock che può essere **disponibile** (nessun thread tiene il lock) o **acquisito** (un thread ha il lock e presumibilmente è nella sezione critica). **Mutex** (Mutua Esclusione) è il nome che la libreria POSIX usa per un lock. Viene utilizzata per la **mutua esclusione** tra thread.

```

1 pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
2
3 pthread_mutex_lock(&lock); // wrapper for pthread_mutex_lock()
4 balance = balance + 1;
5 pthread_mutex_unlock(&lock);

```

↑ Al lock DEI VALORI DI DEFAULT

9.2.1 Metriche per la valutazione dell'implementazione dei lock

Mutua Esclusione: ci dice se il lock sta funzionando, se non c'è mutua esclusione il lock è inutile.

Fairness (equità): i thread che si stanno contendendo per il lock, hanno tutti la stessa possibilità di acquisirlo?

Performance: Qual è l'overhead aggiunto per usare il lock?

9.2.2 Forme di lock

```

1 void lock() {
2     DisableInterrupts();
3 }
4 void unlock() {
5     EnableInterrupts();
6 }

```

↳ PERCHÉ IL CONTINUA OG. PROCESSORE

Problemi: richiede troppa fiducia nelle applicazioni, non funziona con i multiprocessori, perdere gl'interrupt porterebbe a seri problemi.

First attempt: Using a *flag* denoting whether the lock is held or not.

- o The code below has problems.

```

1 typedef struct __lock_t { int flag; } lock_t;
2
3 void init(lock_t *mutex) {
4     // 0 -> lock is available, 1 -> held
5     mutex->flag = 0;
6 }
7
8 void lock(lock_t *mutex) {
9     while (mutex->flag == 1) // TEST the flag
10         ; // spin-wait (do nothing)
11     mutex->flag = 1; // now SET it !
12 }
13
14 void unlock(lock_t *mutex) {
15     mutex->flag = 0;
16 }

```

- o **Problem 1:** No Mutual Exclusion (assume flag=0 to begin)

Thread1	Thread2
<pre> call lock() while (flag == 1) interrupt: switch to Thread 2 </pre>	<pre> call lock() while (flag == 1) flag = 1; interrupt: switch to Thread 1 </pre>
<pre> flag = 1; // set flag to 1 (too!) </pre>	

DUE THREAD LEVA SEZIONE CRITICA

- o **Problem 2:** Spin-waiting wastes time waiting for another thread.

Servono quindi delle istruzioni atomiche supportate dall'hardware. Come **Test and Set**.

```

1  int TestAndSet(int *ptr, int new) {
2      int old = *ptr; // fetch old value at ptr
3      *ptr = new;      // store 'new' into ptr
4      return old;      // return the old value
5  }

```

Questa sequenza di istruzioni viene eseguita atomicamente. Prende due parametri, un puntatore a un intero in memoria e un nuovo valore intero. Ne abbiamo visto l'implementazione con spin-lock, il problema è che non è un metodo equo.

Fetch and Add incrementa un valore atomicamente mentre ritorna il vecchio valore a un particolare indirizzo.

```

1  int FetchAndAdd(int *ptr) {
2      int old = *ptr;
3      *ptr = old + 1;
4      return old;
5  }

```

FETCH DI UN VALORE

GLI AGGIUNGE UN UNITÀ

Fetch-And-Add Hardware atomic instruction (C-style)

Con fetch and add si va a implementare **ticket lock**.

1	<code>typedef struct __lock_t {</code>	Immaginiamo il lock come una biglietteria che
2	<code>int ticket;</code>	vende ticket, ticket mi dice quanti ticket ho
3	<code>int turn;</code>	venduto (quanti thread hanno provato a fare
4	<code>} lock_t;</code>	lock) il turn stabilisce di chi è il turno di entrare
5		nella sezione critica. Quando facciamo lock la
6	<code>void lock_init(lock_t *lock) {</code>	fetch and add ci restituisce il mio turno geust
7	<code>lock->ticket = 0;</code>	ultima infatti va a prendere il numero di ticket
8	<code>lock->turn = 0;</code>	venduti fino a quel momento e mi restituisce il
9	<code>}</code>	mio turno. Ora devo semplicemente aspettare
10		che sia il mio turno. Unlock invece va ad
11	<code>void lock(lock_t *lock) {</code>	aumentare il valore della variabile del turno,
12	<code>int myturn = FetchAndAdd(&lock->ticket);</code>	quindi tocca al thread i+ unesimo
13	<code>while (lock->turn != myturn)</code>	
14	<code> ; // spin</code>	
15	<code>}</code>	
16	<code>void unlock(lock_t *lock) {</code>	
17	<code> FetchAndAdd(&lock->turn);</code>	
18	<code>}</code>	

In alcuni casi queste soluzioni possono essere inefficienti. Quando un thread inizia ad aspettare, potrebbe sprecare un intero quanto di tempo a aspettare e controllare il valore. Per evitare lo spinning abbiamo bisogno dell'aiuto del sistema operativo.

Instead of spinning, **give up the CPU** to another thread.

- o OS system call moves the caller from the *running state* to the *ready state*.

1	<code>void init() {</code>
2	<code> flag = 0;</code>
3	<code>}</code>
4	
5	<code>void lock() {</code>
6	<code> while (TestAndSet(&flag, 1) == 1)</code>
7	<code> yield(); // give up the CPU</code>
8	<code>}</code>
9	
10	<code>void unlock() {</code>
11	<code> flag = 0;</code>
12	<code>}</code>

Lock with Test-and-set and Yield

